Discovery Executive Summary

Project: discovery-pingCRM-7Aug · **Generated:** 07/08/2026, 18:05:55

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 7 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Code Quality & Complexity Analysis	—
3	Frontend Modernization Analysis	—
4	Backend Modernization Analysis	—
5	Testing & Quality Assurance Analysis	—
6	Security Analysis	—
7	Performance & Sustainability Analysis	—

---

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

This is a Laravel 11 + Inertia/React application (PingCRM base) that has been overgrown with a large, deliberately-legacy "IVR" subsystem. Architectural health is **High Risk**: 83 single-action IVR controllers each carry ~759 lines of inline business logic, raw string-concatenated `DB::select` queries, `extract($request->all())`, hard-coded tenant IDs, and `new SomethingGodService()` instantiation — so controllers, persistence, and domain rules are fused into one untestable layer. A Repository tier (12 classes) and a Service tier (12 `*GodService` classes) both exist but are architecturally hollow: the repositories are referenced by zero call-sites (dead abstraction) and the services are god-classes with 45 unrelated public methods, mutable `static` state, and embedded secrets. There is no Dependency Injection wiring at all (`AppServiceProvider` binds nothing; `Model::unguard()` is global), so nothing can be substituted or mocked. On the frontend, 916 components repeat the same pathology in React terms — 374 components issue inline `fetch()` calls to hard-coded URLs, 147 legacy class components mix paradigms, and 8 near-identical 1,101-line "formatter" modules duplicate the same logic. The dominant risks are **change amplification** (one schema or rule change touches dozens of controllers, services, repos and components at once) and **hidden coupling** (reporting reads other domains' operational tables directly), both of which will make any modernization or team scaling extremely costly until bounded contexts, a real service layer, and DI are introduced.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	<span class="rating	Moderate	<span class="rating rating-high-	Measured	Rating

			rating-good">Good		risk">High Risk		
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	**~683 LOC avg** (81 of 91 > 300 LOC; IVR = 759 LOC each)	High Risk
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	**83** controllers with inline DB/model access + rules	High Risk
H3	Missing Repository Pattern	Direct DB access points outside repositories	<10	10–20	>20	**95** files use `DB::` directly; 12 repositories have **0** references	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	**0** cycles observed	Go
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	**13** (5 backend helpers + 8 frontend dup formatters)	High Risk
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	**~9%** compliant (83 of 91 controllers embed raw SQL)	High Risk
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 backend > 1000 LOC, **but** 12 `*GodService` @ **45** methods + static state	High Risk
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	**4** (Reports, IvrHub, LoadsIvrModuleData, IvrAccountContext)	Mod
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	**~21%** (3 of 14 tables shared, no ownership)	Mod
H10 *(additional)*	No Dependency Injection / Service Locator	Services resolved via `new`	0	1–10	>10	**80** `new *GodService()` ; 0 container bindings	High Risk
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	avg **139 LOC**; 134 of 522 pages (26%) > 300 LOC	Mod
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	**374** components with inline `fetch()` /hard-coded URLs	High Risk
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	**1** (`Hub/Index.tsx` = 479 LOC); 134 more in 300–400 band	Mod
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	**≤2** — components self-fetch into local state	Go

F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	**147** class components mixed with function components	High Risk

No additional hotspots beyond H10 were observed.

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Reduce the 83 IVR controllers to validate-and-delegate; extract logic into Application Services	High Risk	Critical
H2 Missing Service Layer	Introduce per-capability Application Services; move query/workflow assembly out of controllers	High Risk	Critical
H6 Direct SQL in Controllers	Move all `DB::` queries out of controllers into bound-parameter repositories; add CI grep gate	High Risk	Critical
H10 No Dependency Injection	Bind interfaces in `AppServiceProvider`, inject services, remove `new *GodService()` and `Model::unguard()`	High Risk	Critical
H3 Missing Repository Pattern	Give repositories interfaces + bound queries; route all persistence through them	High Risk	High
H5 Shared Utility Abuse	Replace 5 `LegacyIvr*` helpers with domain services; collapse 8 duplicate formatter modules into one	High Risk	High
H7 God Classes	Split each `*GodService` into domain service + repository + application service; remove static state/secret	High Risk	High
F2 Missing Frontend Service Layer	Introduce a typed API client + hooks; replace 374 inline `fetch()` /hard-coded URLs	High Risk	High
F5 Legacy Component Patterns	Codemod 147 class widgets to hooks; add error boundaries and a ban-class lint rule	High Risk	High
H8 Domain Boundary Violations	Introduce published read models / ACL; forbid cross-context table reads in Reports & Hub	Moderate	Medium
H9 Shared Database Coupling	Assign per-table ownership; serve reporting via an owned read model instead of live joins	Moderate	Medium
F1 Business Logic in Components	Move validation/transform into shared hooks/schema; split 300–400-LOC page templates	Moderate	Medium
F3 God / Oversized Components	Decompose `Hub/Index.tsx` (479 LOC) and the shared 392-LOC template into widgets + hooks	Moderate	Medium

1.5 Expected Outcomes

- **Testable, reusable business logic** — with Application/Domain Services and constructor DI in place, workflows can be unit-tested with mocked repositories and reused from HTTP, CLI and queued jobs instead of being copy-pasted across 83 controllers.
- **Schema and persistence freedom** — funnelling every `DB::` call through parameter-bound repositories removes the injection holes and lets the team rename/repartition `ivr_*` tables without editing dozens of hand-written query strings.
- **Independent, extractable domains** — bounded contexts with published interfaces and an anti-corruption layer let Queue, Call Routing, Analytics and Reporting evolve (and eventually extract) without silently breaking each other through shared tables.
- **A consistent, maintainable frontend** — a single typed API client plus hooks and function components collapses 374 inline fetches and 147 class widgets into one convention, so endpoint/auth changes are one-file edits and screens stop leaking timers and blanking on errors.

- **Sharply lower change amplification** — de-duplicating the 5 helpers and 8 formatter modules and enforcing CI gates (no `DB::` in controllers, no `React.Component`, duplication check) means a single rule change stops rippling through dozens of near-identical files.","ttft_ms":7882,"ttft_stream_ms":7697,"time_to_request_ms":6140,"type":"result","duration_ms":491456,"uuid":"07a4177b-031e-4e68-a1fd-2230b3ef721b"}

---

2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

This codebase is a Ping CRM base (Laravel + Inertia/React) onto which a very large "Legacy IVR" surface has been grafted, and that surface is dominated by copy-paste rather than genuine algorithmic complexity. Coverage spans **both layers**: 141 backend PHP files and 1,051 frontend TypeScript/TSX files (1,192 total). The most severe findings are structural, not branch-depth: the frontend contains 8 files over 1,100 LOC and 133 near-identical `LegacyPass2_*.tsx` pages, while the backend has 82 fat IVR controllers at exactly 759 LOC each, 12 `*GodService` classes, and 12 near-identical repositories — pushing estimated overall duplication well above 60%. By contrast, per-method cyclomatic complexity is genuinely low (max ≈ 8) and no single function exceeds ~22 LOC, so those hotspots rate Good. Git history was available (124 commits, 2020–2026); churn and defect-fix frequency on the maintained core are low, but note that the entire high-risk IVR surface arrived in a single bulk commit — so its near-zero churn is deceptive, not reassuring. Additional backend anti-patterns were confirmed: shared mutable `public static` state in all 12 God services and ~4,940 uses of `extract()` for dynamic variable creation.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Ra
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	~8 (`IvrHubController::loadStats`)	<s ra
H2	Large Classes	Largest class/file LOC	<300	300–1000	>1000	1,101 LOC (frontend); 759 LOC × 82 (backend)	<s ra ris
H3	Large Functions	Largest function LOC	<50	50–200	>200	~22 LOC (`handleStore`)	<s ra
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~60% (handlers + workflows copied per module)	<s ra ris
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~65% (est., manual — no jscpd/phpcpd configured)	<s ra ris
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	~0.35/mo (top file, 27 changes / 78 mo)	<s ra
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	3 (`Contacts/Index`)	<s ra
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	43–71% (maintained core files)	<s ra

							m
H9	Global Mutable State *(additional)*	Shared mutable static/global holders (target 0)	0	1–3	>3	12 (`public static $sharedRuntimeCache`)	<s ra ris
H10	Dynamic Variable Creation *(additional)*	`extract()` /dynamic-var uses (target 0)	0	1–50	>50	~4,940 `extract($payload)` calls	<s ra ris

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	18	4.50
Code Churn	25%	15	3.75
Defect Density	20%	30	6.00
Class/Function Size	15%	80	12.00
Business Logic Duplication	10%	92	9.20
Developer Ownership Risk	5%	55	2.75
Hotspot Score	**100%**		**38 / 100**

2.5 Actions Required

Hotspot	Action	Rating	Priority
H4 Business Logic Duplication	Consolidate per-module workflows into one `IvrModuleService` + a shared `useIvrRecordForm` hook	High Risk	Critical
H5 Duplicate Code (general)	Replace 133 `LegacyPass2` pages / 8 formatter clones / 12 repos with data-driven components; add `jscpd` / `phpcpd` CI gate	High Risk	Critical
H2 Large Classes	Split 82 fat IVR controllers into single-action controllers + services; collapse >1,000-LOC formatter modules	High Risk	High
H9 Global Mutable State	Replace `public static $sharedRuntimeCache` with request-scoped/per-tenant cache in all 12 God services	High Risk	High
H10 Dynamic Variable Creation	Replace ~4,940 `extract($payload)` calls with explicit access/DTOs; ban `extract()` via PHPStan	High Risk	High
H8 Ownership Issues	Add CODEOWNERS for CRM core and assign an owner/deprecation decision for `Legacy/**` and `Pages/Ivr/**`	Moderate	Medium

2.6 Expected Outcomes

- **Lower maintenance cost:** consolidating duplicated workflows and files removes an estimated ~60% of redundant code, so each business-rule change is made once instead of across 12–230 sites.
- **Fewer regressions:** a single shared service + form hook eliminates the "fixed here but not there" defect class that dominates the IVR surface today.
- **Faster, safer builds:** deleting 8 oversized formatter modules and ~360 clone components shrinks the frontend bundle and CI time; a duplication gate prevents new clones.

- **Correctness & isolation:** removing shared static state and `extract()` closes cross-tenant leakage and restores static-analysis (Larastan) coverage.
- **Clearer ownership:** CODEOWNERS on the maintained core and an explicit decision on the Legacy/IVR surface reduce coordination overhead and stop dead code from accreting.

Report saved to `docs/discovery/02-code-quality-complexity.md` (both frontend and backend layers covered). The orchestration UI will convert it to the matching PDF.","ttft_ms":4694,"ttft_stream_ms":3705,"time_to_request_ms":125,"type":"result","duration_ms":452890,"uuid":"7c0f0161-e69f-4426-b88a-b19daa69a597"}

---

3. Frontend Modernization Analysis

EXECUTIVE SUMMARY

This is the React/Inertia.js single-page frontend of a PingCRM-derived CRM that has been heavily extended with a synthetic "IVR" telephony surface. The core PingCRM slice (Pages/Contacts, Organizations, Users, Auth, Reports and the 14-component `Shared/` library) is clean, idiomatic React 19 with TypeScript strict mode on. The IVR extension, however, is a large modernization liability: 916 component files, of which ~517 are near-identical duplicates (133 `LegacyPass2_*` page clones, 229 `*Monolith*` components, 147 class-based `.jsx` widgets, 8 byte-identical formatter utilities). The most severe findings are the total absence of an API/service layer — all 874 `fetch()` calls are made inline inside components against an **unauthenticated** `ivr-legacy/*` route group — plus 375 files that start a 5-second `setInterval` poll with no cleanup (a systemic memory/network leak), 13,701 inline-`style` occurrences with zero design tokens, and a High-severity lodash CVE in a dependency that the frontend does not even import. TypeScript strict is enabled and ESLint is configured, but ESLint is not run in CI, and there are no error boundaries anywhere in the tree.

§3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~517 / 916 ≈ **56%** near-identical	High Risk
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	**84.0%** (147 class `.jsx`)	Moderate
H3	Massive Components	Largest component LOC	<200	200–500	>500	**479 LOC** (134 files >300)	Moderate
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	**~0%** (no global store)	Good
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	**≤2** (Inertia page props)	Good
H6	Weak Frontend Architecture	Feature modules with	>80%	50–80%	<50%	**<20%** (API+UI+logic mixed)	High Risk

		clean boundaries %					
H7	Missing Component Inventory	Shared component % of total	>30%	15–30%	<15%	**1.5%** (14 / 916, no Storybook)	High Risk
H8	No Design System	Inline-style / magic-value occurrences	0–5	6–20	>20	**13,701** inline styles, 0 tokens	High Risk
H9	Routing Structure Weakness	Protected routes with guards %	100%	80–99%	<80%	**100%** page routes guarded	Good
H10	No API Integration Layer	API calls in service layer %	>90%	70–90%	<70%	**0%** (874 inline `fetch`)	High Risk
H11	Poor Data Caching	Data-fetching points with caching %	>70%	40–70%	<40%	**0%** (no query cache)	High Risk
H12	Weak Frontend Auth	Token storage + routes guarded	httpOnly + 100%	One gap	Both gaps	httpOnly cookie / **data API unguarded**	Moderate
H13	Frontend Security Vulnerabilities	XSS-risk + hardcoded secrets count	0 each	1–3 total	>3 total	**5+** (2 innerHTML, 2 no-SRI, seeded pw)	High Risk
H14	Frontend Performance Gaps	Memoization / render optimization	good	some gaps	none + leaks	**0** memo, 375 interval leaks	Moderate
H15	Browser Compatibility Gaps	Browserslist + polyfills configured	Both present	One missing	Both missing	polyfills / **no browserslist**	Moderate
H16	Frontend Code Quality	ESLint in CI + TypeScript strict	Both Yes	One Yes	Both No	strict / **ESLint not in CI**	Moderate
H17	Technical Debt & Dependencies	Critical/High CVEs found	0	1–3	>3	**1 High** (lodash) + dead deps	Moderate
H18	*(additional)* useEffect Cleanup / Leaks	Effects with timers lacking cleanup	0	1–10	>10	**375** `setInterval` w/o cleanup	High Risk
H19	*(additional)* Missing Error Boundaries	Error boundaries present	≥1 per area	1 global	0	**0** boundaries app-wide	Moderate

§3.4 Actions Required

Hotspot	Action	Rating	Priority
H10 No API Integration Layer	Move 874 inline `fetch` calls into a typed `services/ivrClient` with auth-header/error interceptors	High Risk	Critical

H12 Weak Frontend Auth	Add `auth` middleware to the `ivr-legacy` route group; restrict mutations to POST/PUT/DELETE	Moderate	Critical
H13 Security Vulnerabilities	Add SRI to CDN scripts, remove seeded `password:'secret'`, sanitize pagination HTML	High Risk	Critical
H18 Interval Leaks	Return `clearInterval` cleanup in 375 effects (or migrate to React Query polling)	High Risk	Critical
H1 Component Duplication	Consolidate `LegacyPass2_*`, `*Monolith*` and `legacyFormatters1-8` into parameterised units	High Risk	High
H6 Weak Architecture	Extract service layer + presentational components; inject `tenantId` from auth context	High Risk	High
H7 Missing Inventory	Publish `Shared/` as `@/ui` + Storybook; migrate IVR raw controls onto it	High Risk	High
H8 No Design System	Introduce design tokens; codemod 13,701 inline styles to Tailwind classes	High Risk	High
H11 Poor Data Caching	Adopt React Query with `staleTime` and post-mutation invalidation	High Risk	High
H2 Legacy Class Components	Convert 147 `*ClassWidget.jsx` to typed function components with a shared hook	Moderate	High
H19 Missing Error Boundaries	Add global + per-feature `ErrorBoundary` with fallback UI	Moderate	High
H16 Code Quality	Add ESLint CI job; re-enable `no-explicit-any`; bring `.jsx` under strict tsconfig	Moderate	Medium
H17 Technical Debt	Remove unused vulnerable `lodash` + dead `react-router-dom`; upgrade prettier/eslint	Moderate	Medium
H3 Massive Components	Replace unrolled markup with data maps; extract logic to hooks (`max-lines` rule)	Moderate	Medium
H14 Performance Gaps	Fix interval leaks, memoize heavy tables, add a CI bundle budget	Moderate	Medium
H15 Browser Compatibility	Add `.browserslistrc`; bundle polyfills locally; wire Autoprefixer	Moderate	Low

§3.5 Expected Outcomes

- **A centralized `services/ivrClient` layer** replaces 874 scattered `fetch` calls, giving one place for auth headers, CSRF, base URL, typed responses and consistent error UX — and making the API mockable in tests.
- **Guarding the `ivr-legacy` route group and removing GET mutations** closes an open, unauthenticated data-and-delete surface that currently sits behind an authenticated page shell.
- **Fixing the 375 interval leaks** (or moving to React Query polling) eliminates a compounding memory/network leak, stabilising long sessions and cutting idle backend traffic.
- **Adopting React Query** brings caching, loading/error states and post-mutation invalidation, ending stale-after-save UX and redundant refetching.
- **Consolidating ~517 duplicated clones** into parameterised components/utilities shrinks the tree, kills behaviour drift, and makes a single fix apply everywhere.
- **A documented `@/ui` library + design tokens** replaces 13,701 inline styles with a single source of truth for colour and spacing, restoring visual consistency and making rebrands one-line changes.
- **Converting 147 class widgets to typed hooks** and enforcing ESLint in CI with `strict` typing catches hook-rule and type errors (including the leaks) before they reach `master`.

- **Removing the unused, CVE-carrying `lodash` and dead `react-router-dom`** clears the one High-severity advisory and reduces attack surface and audit noise to zero.
- **Adding error boundaries** turns a full-page blank-out on any render throw into a contained, recoverable per-feature fallback.","ttft_ms":2446,"ttft_stream_ms":1441,"time_to_request_ms":179,"type":"result","duration_ms":487645,"uuid":"15edec16-e292-4579-b37e-0beb5093972f"}

---

4. Backend Modernization Analysis

EXECUTIVE SUMMARY

The repository is a Laravel 11 PingCRM base whose original CRM surface (Contacts, Users, Organizations, Reports) remains clean and idiomatic, but a large synthetic "IVR Enterprise" legacy subsystem has been grafted on top and it concentrates almost every backend anti-pattern in the catalogue. The IVR layer ships 82 fat single-action controllers, 12 ~373-line "God" services holding mutable `public static` state and hard-coded API keys, and 12 unused repository classes that build SQL by string concatenation. Untyped request data is materialised with `extract()` in 92 files, raw SQL is concatenated directly in controllers and repositories (SQL-injection signature), Eloquent models use `$guarded = []` (mass-assignment wide open), and a `config/ivr_legacy.php` file commits a master API key, a Salesforce client secret and a plaintext password. An 80-route `ivr-legacy` API is exposed with **no authentication middleware, no OpenAPI spec, no versioning and no contract tests**, while a hard-coded `tenant_id = 1` breaks multi-tenant isolation (IDOR). Performance is degraded by ~540 synchronous `sleep(1)` calls on the request path, 420 N+1 accessor methods and zero caching. Database schema and migrations are the one bright spot (indexed FKs, reversible `down()` methods). The overall backend rating is **High Risk**, driven by broken access control, injection/secret exposure and the absence of a real service/data layer.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	`extract()` in 92 files (12 services × 45, 82 controllers × ≤55)	High Risk
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	12 `public static $sharedRuntimeCache` in God services	High Risk
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	~0% — 83 controllers call `DB::` directly; repositories unused	High Risk
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	12 God services `new` -instantiated ~4,400× + 5 static helper classes	High Risk
H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	82 IVR controllers with inline rules/SQL	High Risk
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	0% — 80 duplicative `ivr-legacy` routes, GET+POST on each	High Risk

H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% — no OpenAPI, no versioning, no contract tests	High Risk
H8	Weak Application Architecture	Modules following declared architecture %	>80%	50–80%	<50%	<20% — IVR breaks MVC; logic in controllers/models	High Risk
H9	Missing Module Inventory	Circular dependency count	0	1–3	>3	0 circular deps, but 17 dead files (helpers + repos) unreferenced	Moderat
H10	Database Schema Weakness	FK indexes % + migrations with rollback %	Both >90%	One <90%	Both <90%	FKs indexed & constrained; all migrations have `down()`	Good
H11	Middleware Weakness	Required middleware present + ordered %	100%	80–99%	<80%	No security headers, no request-ID logging, no explicit CORS	Moderat
H12	Auth & Authorization Weakness	Protected routes guarded % + hashing algo	100% + bcrypt/argon2	One gap	Both bad	~26% guarded (80 public API routes) + IDOR; hash = bcrypt	High Risk
H13	Backend Security Vulnerabilities	Injection + hardcoded secrets count	0 each	1–3 total	>3 total	SQLi + mass-assignment + `APP_DEBUG=true` + hardcoded secrets (>3)	High Risk
H14	Performance & Caching Gaps	N+1 patterns found	0	1–5	>5	420 N+1 accessors + 540 `sleep(1)` + 0 caching	High Risk
H15	Outdated & Vulnerable Dependencies	Critical/High CVEs found	0	1–3	>3	0 observed (audit not run offline); `roave/security-advisories` present	Good
H16	Secrets & Configuration in Source	Hardcoded secrets / .env committed	0	1–2	>2	15+ hardcoded secrets (config/ivr_legacy.php + 12 service keys)	High Risk
H17	Backend Code Quality	Linter in CI + max cyclomatic complexity	Both good	One gap	Both bad	Linter + PHPStan in CI, but level 1 only + massive duplication/dead code	Moderat
H18	Swallowed Exceptions (additional)	Empty/silent `catch` blocks (target 0)	0	1–20	>20	~4,400 `catch (\\Throwable)` blocks returning `err` and continuing	High Risk

4.4 Actions Required

Hotspot	Action	Rating	Priority
H12 Auth & Authorization	Apply `auth:sanctum` to all 80 `ivr-legacy` routes; replace hard-coded `tenant_id=1` with authenticated tenant + object-level policies	High Risk	Critical

H13 Security Vulnerabilities	Bind all SQL parameters; replace `$guarded=[]` with `$fillable` ; ship `APP_DEBUG=false` ; stop returning exception messages	High Risk	Critical
H16 Secrets in Source	Move `config/ivr_legacy.php` + 12 service keys to secrets manager and rotate; add secret-scanning to CI	High Risk	Critical
H1 Dynamic Variable Creation	Replace `extract()` in 92 files with typed Form Request DTOs and explicit field access	High Risk	Critical
H3 Direct SQL Outside Data Layer	Route all `DB::` access through repositories with bound params; add arch rule banning `DB::` in controllers	High Risk	High
H5 Missing Service Layer	Extract cohesive domain services; thin the 82 controllers to DTO→service→response	High Risk	High
H4 Static / Singleton Abuse	Bind services in container; inject instead of `new` ; collapse static helpers	High Risk	High
H2 Global Mutable State	Remove `public static $sharedRuntimeCache` ; use scoped cache	High Risk	High
H14 Performance & Caching	Delete `sleep(1)` (queue it); eliminate N+1 accessors; add Redis + `Cache-Control`	High Risk	High
H18 Swallowed Exceptions	Remove blanket `catch(\\Throwable)` that swallows/leaks; log+rethrow with correlation IDs	High Risk	High
H6 API Sprawl	Collapse GET+POST duplicate routes into versioned RESTful resources	High Risk	High
H7 API Governance	Add OpenAPI spec, Spectral linting, contract tests and `/v1` versioning in CI	High Risk	High
H8 Weak Architecture	Enforce HTTP→domain→data layering; move model queries to repositories; add arch tests	High Risk	High
H11 Middleware Weakness	Add security-headers + correlation-id logging; explicit CORS; remove IP auth-bypass	Moderate	Medium
H9 Module Inventory	Delete/archive 17 unreferenced dead files; document module public APIs	Moderate	Medium
H17 Code Quality	Raise PHPStan past level 1; add duplication/complexity gates; collapse cloned methods	Moderate	Medium

4.5 Expected Outcomes

- **Access control restored:** guarding the 80 `ivr-legacy` routes and deriving tenant from the authenticated user eliminates the unauthenticated-mutation and IDOR exposure that currently lets any caller read/write any tenant's data.
- **Injection eliminated:** parameter binding across controllers and repositories, plus `$fillable` allow-lists, removes the SQL-injection and mass-assignment surface (OWASP A03/A08).
- **Credentials secured:** moving 15+ hardcoded secrets to a vault and rotating them prevents lateral movement into Salesforce and blocks future leakage via CI secret scanning.
- **Typed, traceable data flow:** replacing `extract()` with Form Request DTOs makes request handling type-checkable and lets PHPStan rise above level 1.
- **Reusable domain layer:** a real service + repository layer lets IVR logic be shared across HTTP, CLI and queued jobs, and shrinks the 82 fat controllers to thin delegators.
- **Performance headroom:** removing 540 blocking `sleep(1)` calls, eliminating 420 N+1 accessors and adding Redis caching frees the worker pool and cuts database load under concurrency.

- **Observable failures:** removing blanket `catch(\\Throwable)` swallowing and adding correlation-ID logging surfaces production errors to monitoring instead of hiding them behind HTTP 200s.
- **Contract stability:** an OpenAPI spec, versioned routes, API linting and contract tests in CI stop breaking changes from reaching integrators undetected.","ttft_ms":1529,"ttft_stream_ms":1350,"time_to_request_ms":152,"type":"result","duration_ms":435400,"uuid":"11344f34-6955-4959-856a-52c4750ebfb4"}

---

5. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

The test suite is effectively absent relative to the size of the codebase: **4 test files** guard **~1,045 source files** (141 PHP, ~904 TS/TSX). Backend testing is limited to two Inertia feature tests (`ContactsTest`, `OrganizationsTest`) plus one placeholder `ExampleTest` that only asserts `true`; the frontend has a single `smoke.test.ts` that asserts `true === true` and covers zero React components. The entire legacy IVR subsystem — 83 fat controllers, 12 "God" services, 12 repositories, and ~84 unversioned JSON API endpoints under `/api/ivr-legacy` — ships with **no tests whatsoever**, including authentication (`AuthenticatedSessionController`) and crypto helpers (`LegacyIvrCrypto`). Estimated overall coverage is **<5%** (backend ~3%, frontend 0%) based on the test-file-to-source ratio, since no coverage report is present. CI (`.github/workflows/tests.yml`) does run `php artisan test` on every push/PR — a genuine backend gate — but never invokes `npm run test`, so frontend tests are unenforced. There are no integration tests around the service/repository boundaries, no contract tests for the public API, and no end-to-end tests at all.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	Auth + 12 God services + 12 repos + Users/Reports/Images (>25 modules)	High Risk
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	~3% BE · 0% FE (est.)	High Risk
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	~~2 of ~40 boundaries~~ (5%)	High Risk
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% of ~84 `/api/ivr-legacy` endpoints	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0	Good
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	BE gated; FE vitest never runs	Moderate

H7	No End-to-End Tests *(additional)*	E2E specs for critical journeys (target ≥1 suite)	≥1 suite	partial	none	0 (no Cypress/Playwright)	High Risk
H8	Assertion-free Placeholder Tests *(additional)*	Tests with no meaningful assertion (target 0)	0	1–3	>3	2 (`ExampleTest` , `smoke.test.ts`)	Moderate

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Untested Critical Logic	Add PHPUnit unit/feature tests for auth, the 12 God services/repositories, and legacy helpers; add a Login.tsx component test	High Risk	Critical
H2 Low Test Coverage	Enable PHPUnit + Vitest coverage reporting, then drive backend to 75–80% and stand up frontend component tests	High Risk	Critical
H3 Missing Integration Tests	Add RefreshDatabase integration tests across controller→service→repository→DB for IVR modules	High Risk	High
H4 Missing Contract Tests	Add PHPUnit contract tests (status + assertJsonStructure) for the ~84 `/api/ivr-legacy` endpoints	High Risk	High
H7 No End-to-End Tests	Add Playwright with a login + core-journey spec and wire it into CI	High Risk	High
H6 No CI Test Gate (frontend)	Add `npm run test` to `tests.yml` and make it a required check	Moderate	Medium
H8 Assertion-free Placeholder Tests	Replace `ExampleTest` and `smoke.test.ts` with real assertions; add a check flagging assertion-free tests	Moderate	Medium

5.5 Expected Outcomes

- **Critical paths protected before modernization** — auth, the 12 God services/repositories, and legacy crypto/string helpers gain characterization and unit tests, so refactors and extractions can be verified against a regression net instead of shipping blind.
- **A real coverage baseline replaces guesswork** — enabling PHPUnit and Vitest coverage turns the estimated <5% into a measured number and lets the team track progress toward the 75–80% target per layer.
- **IVR wiring is verified end to end** — integration tests across controller→service→repository→DB catch broken seams that unit tests miss, especially in sync/import flows.
- **The public API stops breaking silently** — contract tests on `/api/ivr-legacy` fail the build when a response schema or status changes, protecting the React client and any external consumers.
- **CI enforces quality on both layers** — adding the frontend Vitest step (and Playwright E2E) as required checks means every PR runs the full suite, so tests actually prevent regressions rather than merely existing.

The Markdown deliverable is complete at `docs/discovery/05-testing-and-quality-assurance.md` ; the orchestration UI will convert it to the PDF automatically.","ttft_ms":2441,"ttft_stream_ms":1537,"time_to_request_ms":241,"type":"result","duration_ms":254906,"uuid":"1422492b-faa6-400f-932d-7b41cd194d0c"}

---

6. Security Analysis

EXECUTIVE SUMMARY

This review covered **both layers**: the PHP/Laravel backend (141 source files) and the React/Inertia TypeScript frontend (904 files). The posture is **High Risk**, driven by an extensive "Legacy IVR" subsystem grafted onto an otherwise-idiomatic PingCRM base. The most severe issues are **unauthenticated SQL injection** — 80 IVR controllers build `SELECT ... LIKE '%\".$q.\"%'` strings directly from request input and 12 legacy repositories run 480 hand-concatenated `DB::select($sql)` queries — reachable through **80** `ivr-legacy/*` **API routes with no** `auth` **middleware** that accept **GET for state-changing actions**. On top of that, **4,400 controller endpoints call** `extract($request->all())`, and every legacy controller hard-codes `tenantId = 1`, collapsing multi-tenant isolation. 12 "God services" embed hard-coded API keys. Frontend risk is narrower: `Shared/Pagination.tsx` renders server-supplied labels through `dangerouslySetInnerHTML`. Dependency hygiene is mixed — `roave/security-advisories` guards Composer, but there is **no audit/Dependabot step in CI**, and npm pins EOL majors.

6.1 Security Benchmark Ratings

#	Security KPI	Target	Good	Moderate	High Risk	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	≥2 (unauth SQLi; mass-assignment via `extract`)	High Risk
H2	High Vulnerabilities	0	<5	5–10	>10	>10 (broken access control, IDOR, GET state-change, hard-coded secrets)	High Risk
H3	Medium Vulnerabilities	low	<20	20–50	>50	>50 (info-leak error swallowing, XSS sink, misconfig)	High Risk
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	~3.2/KLOC (562 sinks); ~29/KLOC incl. `extract`	High Risk
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	~20% clean (8/10 categories with findings)	High Risk
H6	Critical/High Vulnerable Deps	0	0	1	>1	Unverified — no scanner; ≥1 suspected (EOL majors)	Moderate
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~12% (react-router-dom 5, ESLint 8, Prettier 2)	Moderate

H8	End-of-Life Dependencies	0	0	1–5	>5	3 (react-router-dom 5.2.0, ESLint 8, Prettier 2.x)	Moderate

6.5 Actions Required

Finding	Action	Rating	Priority
SQL injection (80 controllers + 480 repo queries)	Parameterize all queries / use Eloquent; add CI grep gate on raw concatenation	High Risk	Critical
`extract($request->all())` variable injection & mass assignment (4,400)	Remove `extract` ; use `$request->validate()` allow-list + `$fillable` create()	High Risk	Critical
Broken access control — 80 unauth GET state-change routes; hard-coded tenant	Add `auth` middleware, restrict verbs, derive tenant from user, add ownership policies	High Risk	Critical
Hard-coded secrets & fake `LegacyIvrCrypto`	Move keys to env, rotate, replace with `Crypt::encryptString` ; add secret scanning	High Risk	High
Security misconfiguration — verbose errors, `APP_DEBUG=true` , no headers	Log errors server-side, disable debug in prod, add CSP/HSTS/X-Frame-Options middleware	Moderate	Medium
Frontend XSS sink (FS1) `dangerouslySetInnerHTML`	Render labels as text / DOMPurify; enable `react/no-danger`	Moderate	Medium
Vulnerable/EOL npm deps & no dependency scan (FS4)	Upgrade/remove EOL deps; add `npm audit` + `composer audit` + Dependabot	Moderate	Medium
Logging/monitoring gap on state-changing endpoints	Add audit logging + alerting for auth/access events; stop swallowing exceptions	Moderate	Medium
Data-integrity — unpinned dev-stability deps, auto-synced routes	Pin stable versions; review generated route file in PRs	Moderate	Medium

The full report — including §6.2 evidence with code excerpts and affected-files directives, and the three Mermaid diagrams in §6.4 — is in `docs/discovery/06-security.md` , ready for the orchestration UI to render to PDF.","ttft_ms":1508,"ttft_stream_ms":1334,"time_to_request_ms":82,"type":"result","duration_ms":338090,"uuid":"7c0bef7f-514a-4519-8743-55cc4cd0b098"}

---

7. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

PingCRM's core CRM surface (Contacts, Organizations, Reports, IVR module views) is written well — the live read paths in `ReportsController` and the `LoadsIvrModuleData` trait use indexed, joined, `limit()` -bounded queries. The runtime-performance risk is concentrated entirely in a bolted-on "legacy IVR monolith" surface: twelve `*GodService` classes expose 540 workflow methods that each call `sleep(1)` as a synthetic "blocking synchronous remote sync", and each is wired to a live HTTP route through 4,400 thin `legacyEndpoint*` methods. Those blocking calls hold PHP-FPM/Apache workers for the full request, starving the worker pool, while an unbounded static `$sharedRuntimeCache` and 480 `SELECT *` repository methods plus 94 unbounded `->get()` calls load whole tenant tables into memory and Inertia payloads. The dominant hotspots are API latency (P3), memory retention (P4), and worker-pool concurrency (P6) — all High Risk. Sustainability posture is partial: an always-on Apache web process with no autoscaling

wastes worker-seconds and energy on artificial sleeps, and the CI pipeline caches Composer but not npm and rebuilds all assets on every push.

7.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	0 nested-loop/quadratic sites	<span class= rating-good">
P2	Database Performance	Deferred → Backend Modernization (H14/H10)	—	—	—	See Backend Modernization	— (deferred)
P3	API Performance	Response-latency hotspots	0	1–5	>5	540 blocking `sleep(1)` + 174 unbounded reads	<span class= rating-high-risk">High Ri
P4	Memory Efficiency	High-memory sites	0	1–3	>3	12 static caches + 480 `SELECT *` + 94 `->get()`	<span class= rating-high-risk">High Ri
P5	CPU Efficiency	CPU-intensive operations	0	1–5	>5	0 (helpers are string appends, no real crypto/hashing)	<span class= rating-good">
P6	Concurrency	Parallelizable work + pool sizing (blocking-I/O → Backend Modernization H14)	0	1–5	>5	540 worker-blocking sites, no queue offload, no pool config	<span class= rating-high-risk">High Ri
P7	Caching	Deferred → Backend Modernization H14 / Frontend Modernization H11	—	—	—	See those reports	— (deferred)
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	N/A — no container/k8s/Terraform/serverless config (Procfile only)	<span class= rating-good">
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	12 modules split into ~45 sequential endpoint calls each + oversized Inertia payloads	<span class= rating-moderate">M
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	Composer cached; npm/node_modules **not** cached; full `vite build` + `--ssr` every push	<span class= rating-moderate">M

P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	1 `Log::` call total; none in hot loops	
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	Always-on Apache dyno, no autoscaling, 540 artificial `sleep(1)` waste worker-seconds	M

No additional hotspots beyond the standard set were observed.

7.5 Actions Required

Hotspot	Action	Rating	Priority
P3 API Performance	Remove artificial `sleep(1)` from all 540 workflow methods; paginate/column-project the `handleStore` / `handleSync` reads	High Risk	Critical
P4 Memory Efficiency	Make `$sharedRuntimeCache` request-scoped or bounded-TTL cache; add LIMIT + column lists to 480 `fetchChunk*` ; paginate 94 `->get()` sites	High Risk	High
P6 Concurrency	Dispatch the 45 independent workflows as a queued `Bus::batch` ; right-size worker/DB pools after sleeps are removed	High Risk	High
P9 Network Efficiency	Coalesce the ~45 per-module endpoints into one batch endpoint; paginate list payloads; enable gzip/brotli at the edge	Moderate	Medium
P10 Build Efficiency	Add npm/node caching to `tests.yml` ; cache or skip the Vite build in the PHP test job	Moderate	Medium
P12 Sustainability	Add a queue worker + autoscaling; remove artificial sleeps so capacity maps to demand; adopt off-peak/carbon-aware scheduling	Moderate	Medium

7.6 Expected Outcomes

- **Removing the 540 `sleep(1)` calls** eliminates a hard 1s-per-endpoint latency floor and frees the worker pool, cutting P95 latency and multiplying achievable concurrency on the same hardware.
- **Bounding the static cache and adding LIMIT/pagination** (480 `SELECT *` + 94 `->get()`) makes peak memory scale with page size instead of table size, removing OOM risk and GC pressure as IVR data grows.
- **Queuing the 45-way sequential fan-out as a `Bus::batch`** turns a ~45s blocking request into an immediate enqueue-and-return, restoring throughput under concurrency and letting independent work run in parallel.
- **Coalescing the chatty per-module endpoints and enabling compression** cuts round-trips and bytes-on-the-wire, lowering tail latency, bandwidth cost, and energy.
- **Caching npm/build in CI, adding autoscaling and a queue worker, and removing the artificial sleeps** shortens the feedback loop and lets the always-on web tier scale down at off-peak — reducing cloud cost and carbon footprint.

The orchestration UI will now convert the saved Markdown to `docs/discovery/07-performance-sustainability.pdf` .","ttft_ms":3476,"ttft_stream_ms":2497,"time_to_request_ms":115,"type":"result","duration_ms":362757,"uuid":"05bd0f6a-02b7-4f42-9799-78cc4bfd216a"}